

Unidad 4
Clase 8

Replicación y Particionamiento

ESCALABILIDAD, REPLICACIÓN Y TEOREMA CAP EN NOSQL

Evaluar distintas estrategias de replicación, particionamiento y escalamiento horizontal, aplicando el teorema CAP para tomar decisiones fundamentadas sobre arquitecturas NoSQL.

UADE

2026

Academic Architectural Blueprint



El paradigma NoSQL

Arquitecturas diseñadas para resolver las limitaciones de modelos centralizados ante el crecimiento masivo de datos estructurados y no estructurados.

Premisa de Escalabilidad

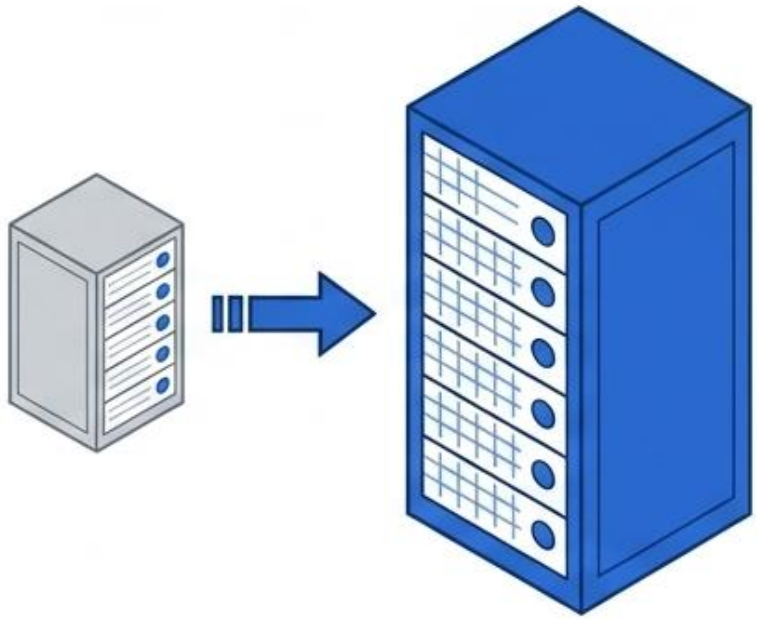
Escalar no es únicamente almacenar más datos. Es la capacidad técnica de sostener el rendimiento, la disponibilidad, la recuperación y la confiabilidad a medida que crecen:

- ✓ Usuarios concurrentes en el sistema.
- ✓ Volumen de consultas y escrituras por segundo.
- ✓ Complejidad operativa de la red distribuida.

Casos de uso aplicados:	
1. Plataformas globales de e-commerce.	2. Sistemas de auditoría continua.
3. Procesamiento de eventos en tiempo real.	4. Redes de Internet de las Cosas (IoT).

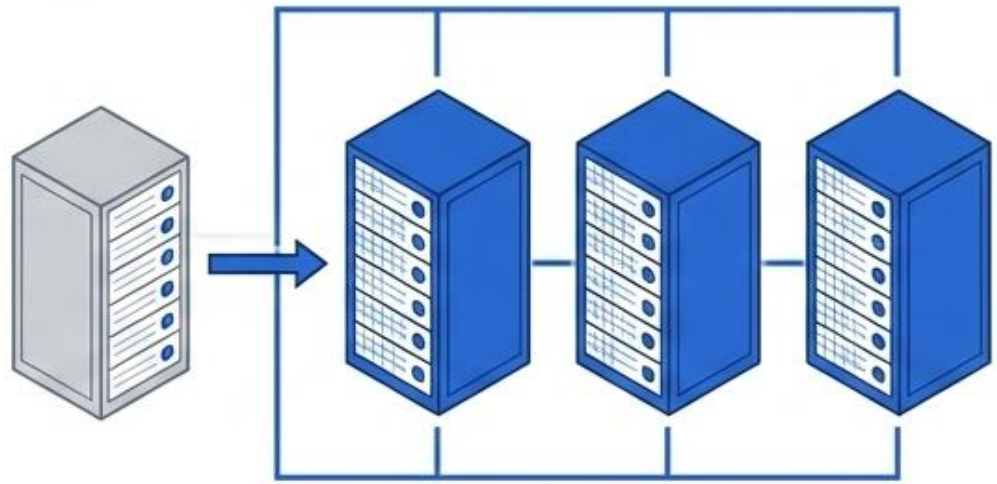
Escalabilidad vertical vs. escalabilidad horizontal

Escalabilidad Vertical (Scale-up)



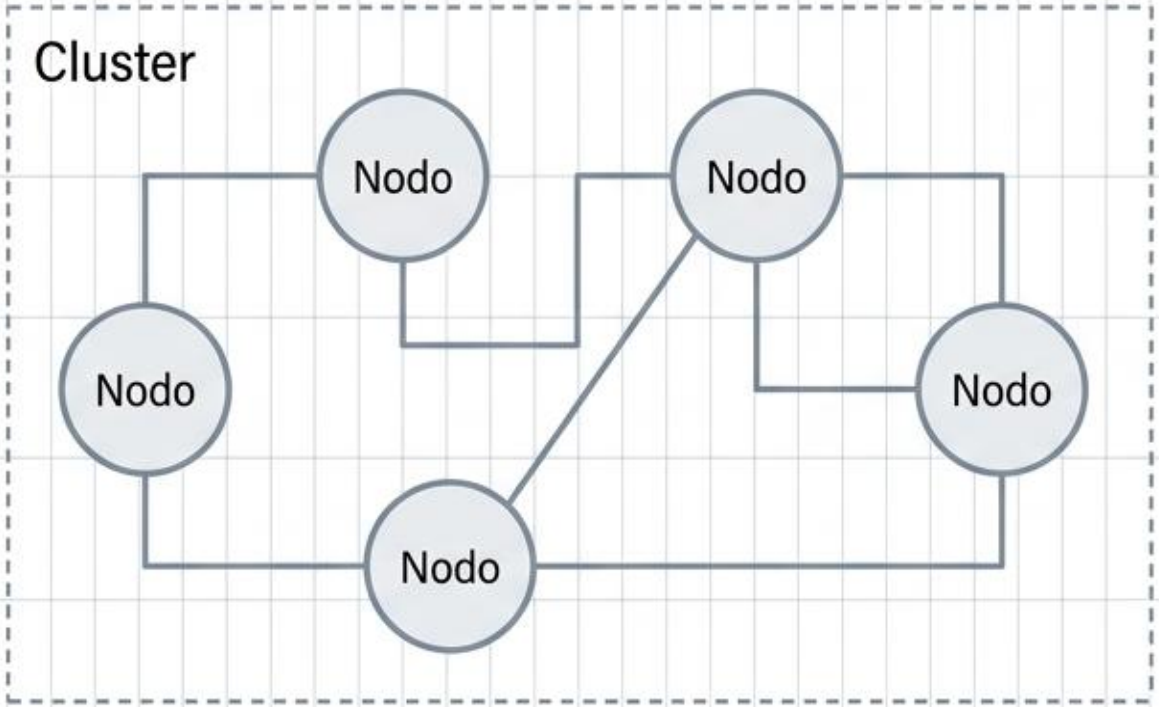
- **Definición:** Aumentar los recursos físicos de un mismo servidor central (añadir CPU, memoria RAM o disco).
- **Ventajas:** Arquitectura centralizada, simple de administrar.
- **Límites:** Tope físico ineludible, altos costos marginales y creación de un Punto Único de Fallo (Single Point of Failure). ❌

Escalabilidad Horizontal (Scale-out)



- **Definición:** Agregar nuevos nodos al sistema para distribuir la carga, los datos y el procesamiento computacional.
- **Ventajas:** Crecimiento virtualmente infinito y tolerancia a fallos mediante redundancia.
- **Vínculo NoSQL:** Los motores NoSQL están diseñados nativamente para aprovechar y orquestar este modelo.

Cluster, nodo y distribución de carga

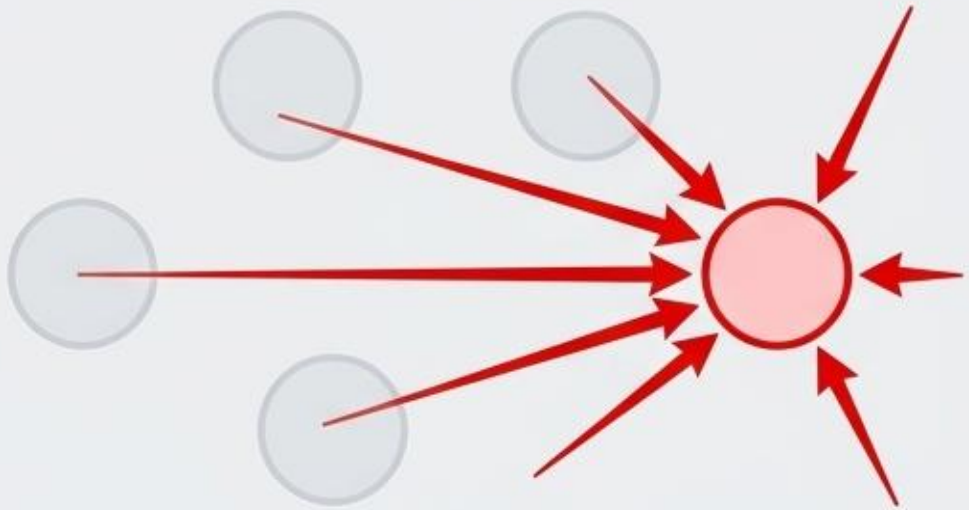


Nodo: Instancia, servidor físico o máquina virtual individual que participa activamente en la red.

Cluster: Conjunto de nodos coordinados que actúan como un único sistema integrado, habilitando la arquitectura distribuida.

Distribución de carga: Asignar roles, fragmentos de datos y operaciones de forma equitativa entre los nodos.

El problema del **Hot Spot** (Zona Caliente)

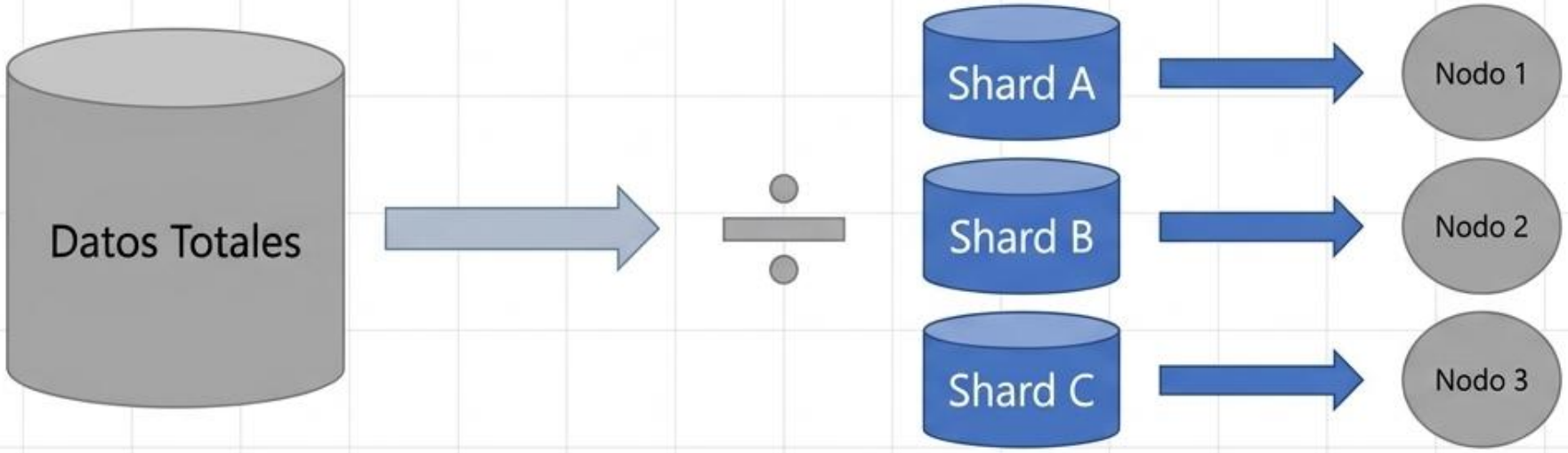


Concentración excesiva y anómala de actividad (lecturas/escrituras) en un solo nodo o partición.

Causa: Diseño de distribución deficiente.

Consecuencia: Transforma un clúster distribuido en un cuello de botella físico, anulando los beneficios del escalamiento.

Particionamiento y sharding



Concepto Central: Técnica matemática que divide un conjunto masivo de datos en partes más pequeñas para distribuirlas equitativamente entre distintos nodos.

- **Shard**: Porción o fragmento resultante del conjunto total tras aplicar la división.
- **Shard Key (Clave de particionamiento)**: Criterio, función hash o campo elegido para determinar en qué nodo exacto residirá cada fragmento.

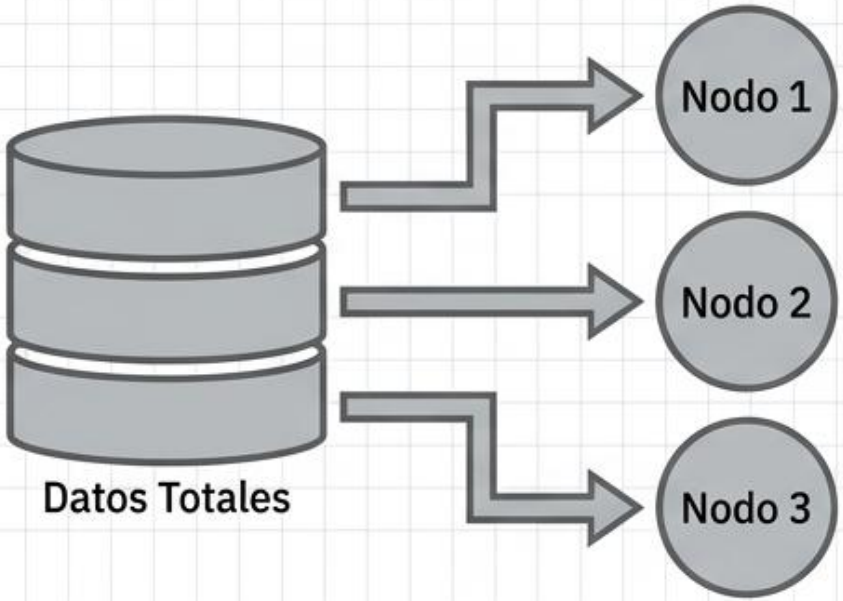
Ejemplos de Criterios:

- Por ID de usuario.
- Por región geográfica.
- Por fecha o tipo de evento.

Impacto Arquitectónico: Elegir una mala clave (por ejemplo, particionar por fecha actual cuando toda la actividad ocurre "hoy") anula la distribución, genera un hot spot inmediato y degrada el rendimiento general.

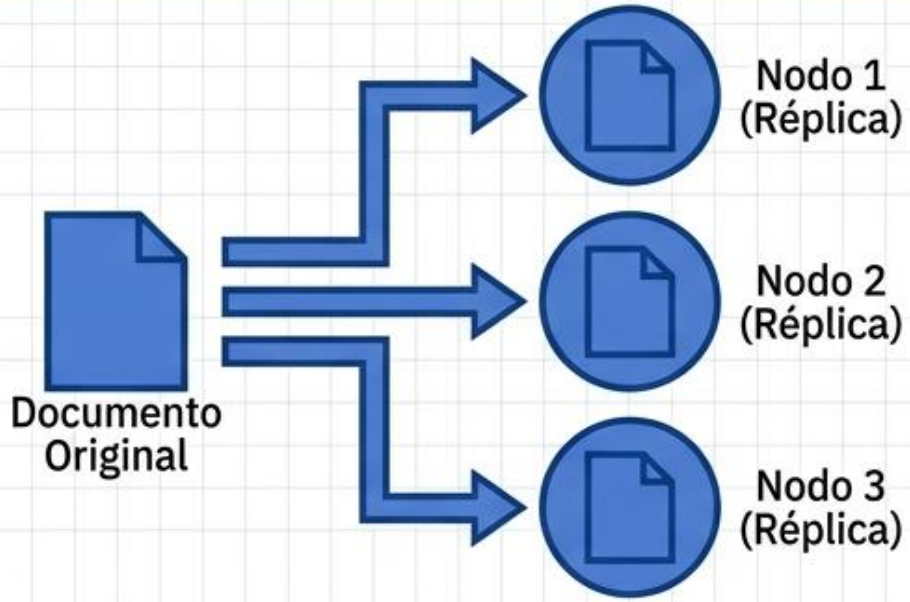
Replicación y factor de replicación

Particionar = Dividir



Divide los datos por razones de volumen y capacidad física.

Replicar = Copiar



Copia los datos por razones de seguridad, disponibilidad y redundancia.

Concepto Central: Mantener copias exactas y sincronizadas de la información en más de un nodo.

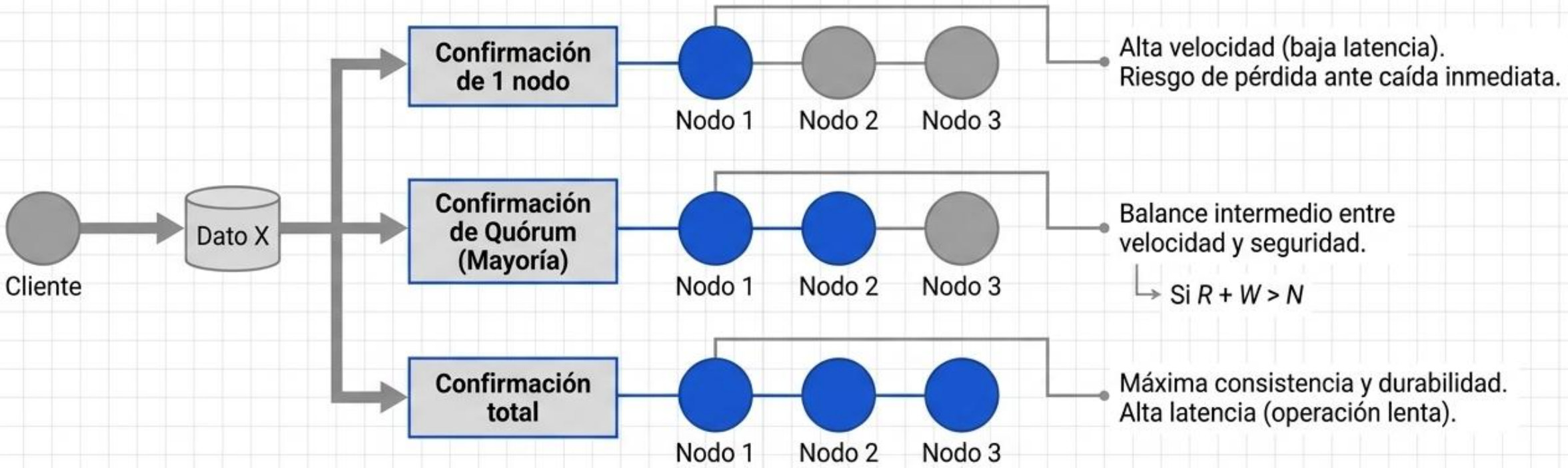
Objetivos Arquitectónicos:

- Maximizar la disponibilidad del servicio.
- Asegurar la recuperación ante fallos de hardware.

Factor de Replicación: Cantidad total de copias independientes mantenidas en todo momento (Ej: Factor 3 = 1 registro original + 2 réplicas).

Replicación, escritura y sincronización

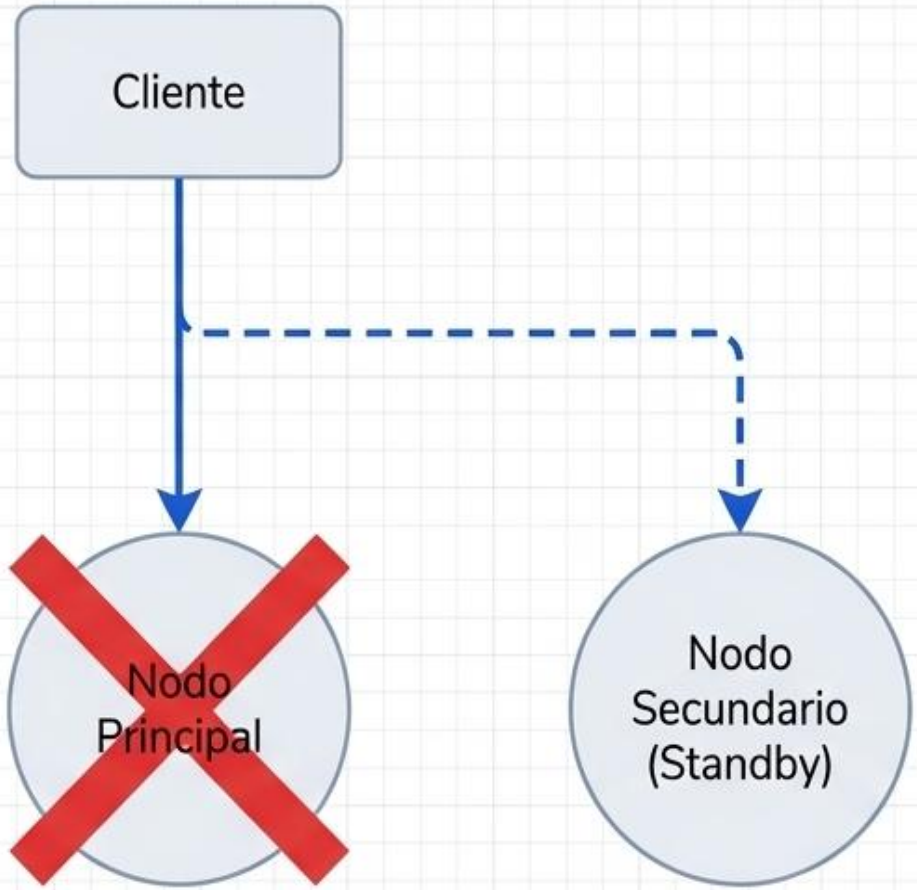
El dilema de la escritura distribuida: Cuando existen múltiples copias, el sistema debe decidir matemáticamente cuándo considera que una escritura fue exitosa.



El Trade-off Arquitectónico:
Exigir sincronización total aumenta la durabilidad, pero penaliza drásticamente la latencia.

Implementaciones en la industria:
- MongoDB utiliza la directiva configurable Write Concern.
- Cassandra permite establecer un Nivel de Consistencia dinámico por operación.

Alta disponibilidad, recuperación y failover



Alta Disponibilidad (High Availability - HA):
Capacidad estructural de continuar procesando peticiones a pesar de sufrir fallos parciales de hardware o red.

Failover (Conmutación por error):
Mecanismo de detección de fallos que transfiere automáticamente el tráfico del nodo caído hacia un nodo sano.

Aclaración Conceptual Crítica

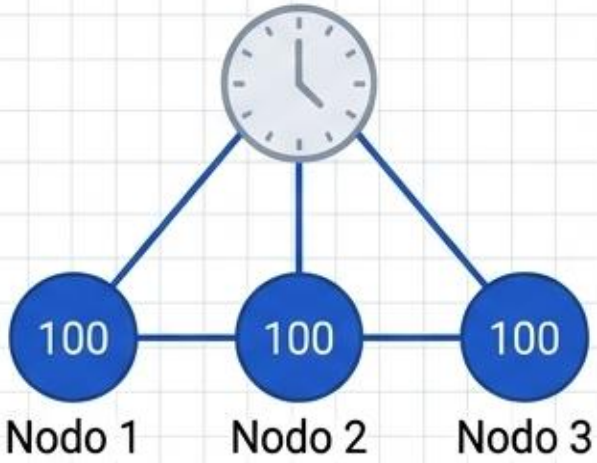
Estar disponible (responder sin error) NO es sinónimo de recuperación perfecta (cero pérdida de datos) ni de consistencia (poseer el último dato exacto).

Sistema de pagos:	→	Prioriza consistencia matemática y recuperación exacta sobre la disponibilidad continua.
Motor de recomendaciones:	→	Prioriza la disponibilidad continua; tolera usar datos levemente atrasados.

Consistencia fuerte y consistencia eventual

Consistencia: El grado de coherencia y exactitud de la información entre todas las réplicas en un momento específico.

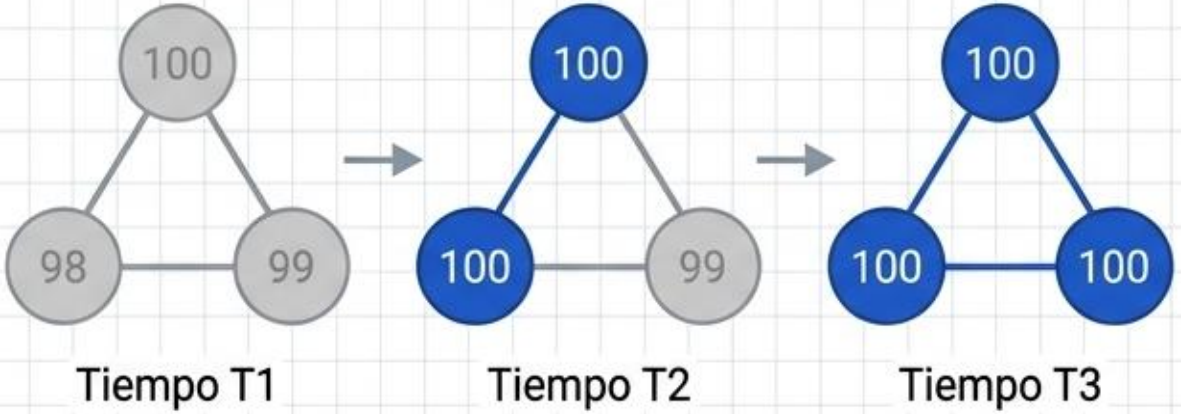
Consistencia Fuerte (Strong)



Mecanismo: Tras confirmarse una escritura, cualquier lectura en cualquier nodo devolverá exactamente el dato más actualizado.

Caso de uso: Saldo de una cuenta bancaria. Un retiro en efectivo debe impactar de inmediato en toda la red.

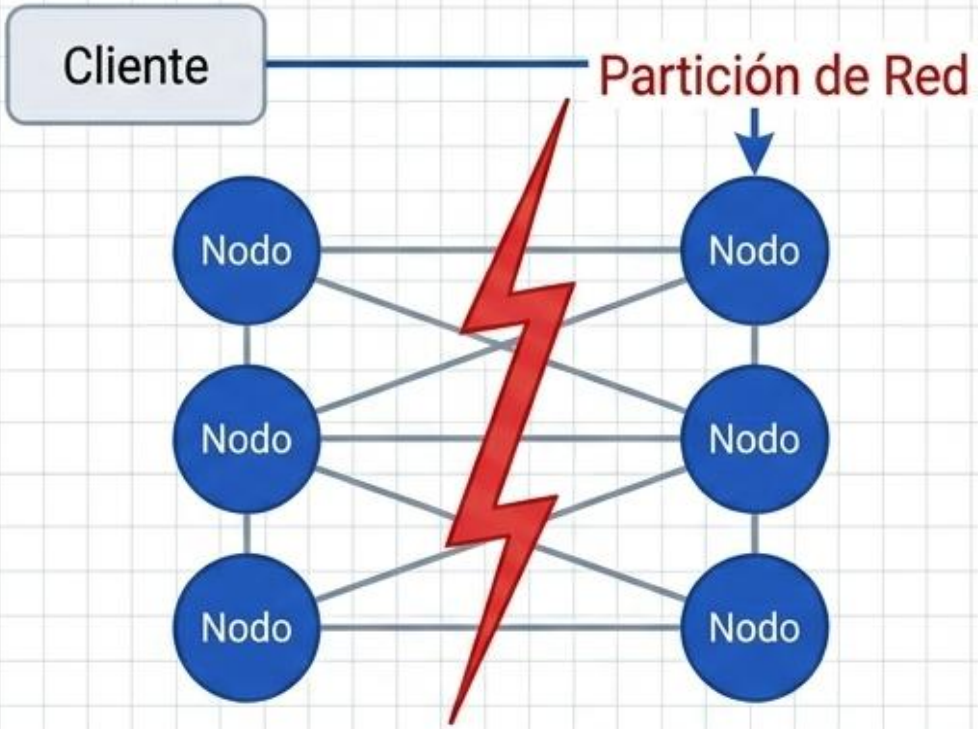
Consistencia Eventual (Eventual)



Mecanismo: Acepta divergencias temporales. Las réplicas difieren temporalmente, pero convergen matemáticamente al mismo estado con el tiempo.

Caso de uso: Contador de 'Me gusta' en redes sociales. Tolerable que usuarios vean cifras diferentes por segundos.

Disponibilidad y tolerancia a particiones



Disponibilidad (Availability):
Capacidad de cualquier nodo operativo para procesar y responder sin errores a las peticiones, incluso si el dato no es el más reciente.

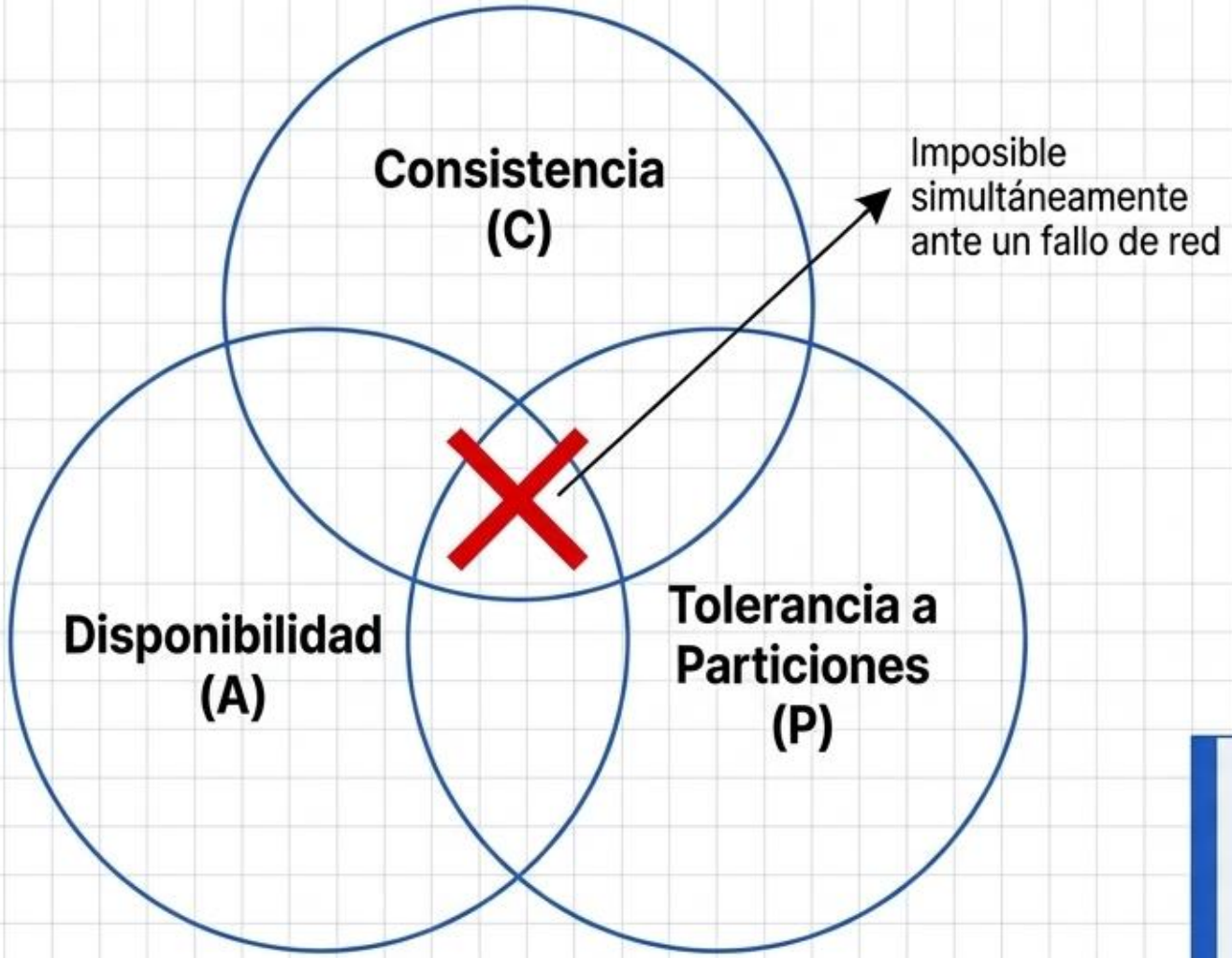
Tolerancia a Particiones (Partition Tolerance):
Capacidad de la arquitectura para seguir prestando servicio a pesar de fallas de comunicación (pérdida de paquetes, cortes de enlace) que aíslen temporalmente los nodos.

El Dilema Arquitectónico ante la Partición

En sistemas distribuidos, una partición no implica servidores apagados, sino servidores incapaces de "hablar". El sistema es forzado a decidir:

1. **Seguir respondiendo** (arriesgando entregar información inconsistente).
2. **Detener** el servicio y **bloquear** respuestas (protegiendo la exactitud).

Teorema CAP



Postulado Fundamental (Eric Brewer):

Demuestra matemáticamente que en un sistema de base de datos distribuido es imposible garantizar las siguientes tres propiedades simultáneamente ante una falla de red:

- **C - Consistencia**: Todos los nodos visualizan la misma data en el mismo instante.
- **A - Disponibilidad:** Toda solicitud recibe una respuesta garantizada.
- **P - Tolerancia a Particiones**: El sistema opera a pesar de cortes de red.

La Regla de Oro Práctica:

Dado que la infraestructura de red es inherentemente imperfecta, la Tolerancia a Particiones (P) se asume como un hecho obligatorio. Por lo tanto, durante un incidente, la arquitectura obliga a elegir entre priorizar Consistencia (C) o Disponibilidad (A).

Plano Arquitectónico Académico

Arquitecturas CP vs. Arquitecturas AP

Premisa de decisión: Cuando ocurre una partición de red...

Arquitecturas CP
(Consistencia + Tolerancia)

- **Comportamiento ante falla:**
  Bloquea nuevas escrituras o rechaza directamente las peticiones si no puede asegurar sincronización total (sacrifica Disponibilidad).
- **Objetivo Crítico:**
  Evitar absolutamente la generación de versiones conflictivas del mismo registro.
- **Escenario Ideal:**
  Entornos financieros, sistemas de pagos transaccionales estrictos.

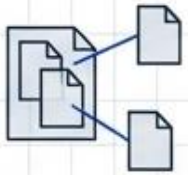
Arquitecturas AP
(Disponibilidad + Tolerancia)

- **Comportamiento ante falla:**
  El clúster continúa operando en ambos lados de la red aislada, devolviendo la mejor información que posee (sacrifica Consistencia exacta).
- **Objetivo Crítico:**
  Mantener el negocio en línea ininterrumpidamente; reconciliar los datos cuando la red vuelva.
- **Escenario Ideal:**
  Redes sociales, plataformas de contenido, e-commerce de consumo masivo.

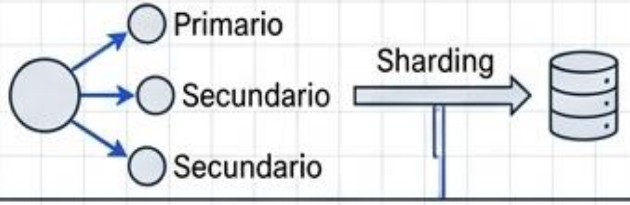
Aplicación de arquitecturas: MongoDB y Cassandra

MongoDB (Motor Orientado a Documentos)

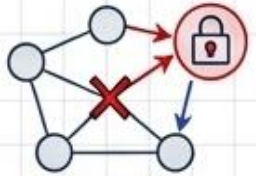
Estructura:
Almacena datos en formatos flexibles tipo BSON (JSON binario) ideal para jerarquías variables.



Distribución:
Opera mediante Replica Sets (un primario, múltiples secundarios) para redundancia, y Sharding para particionamiento horizontal.

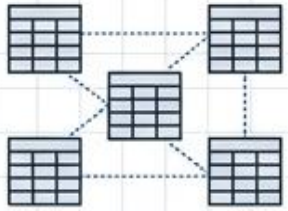


Comportamiento CAP:
Tiende a CP. Ante una desconexión severa, el clúster detiene escrituras hasta completar la elección de un nuevo líder, protegiendo consistencia.

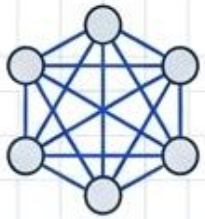


Cassandra (Motor de Familias de Columnas)

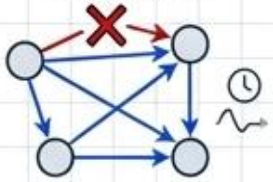
Estructura:
Diseñada para distribuir volúmenes masivos de datos tabulares distribuidos.



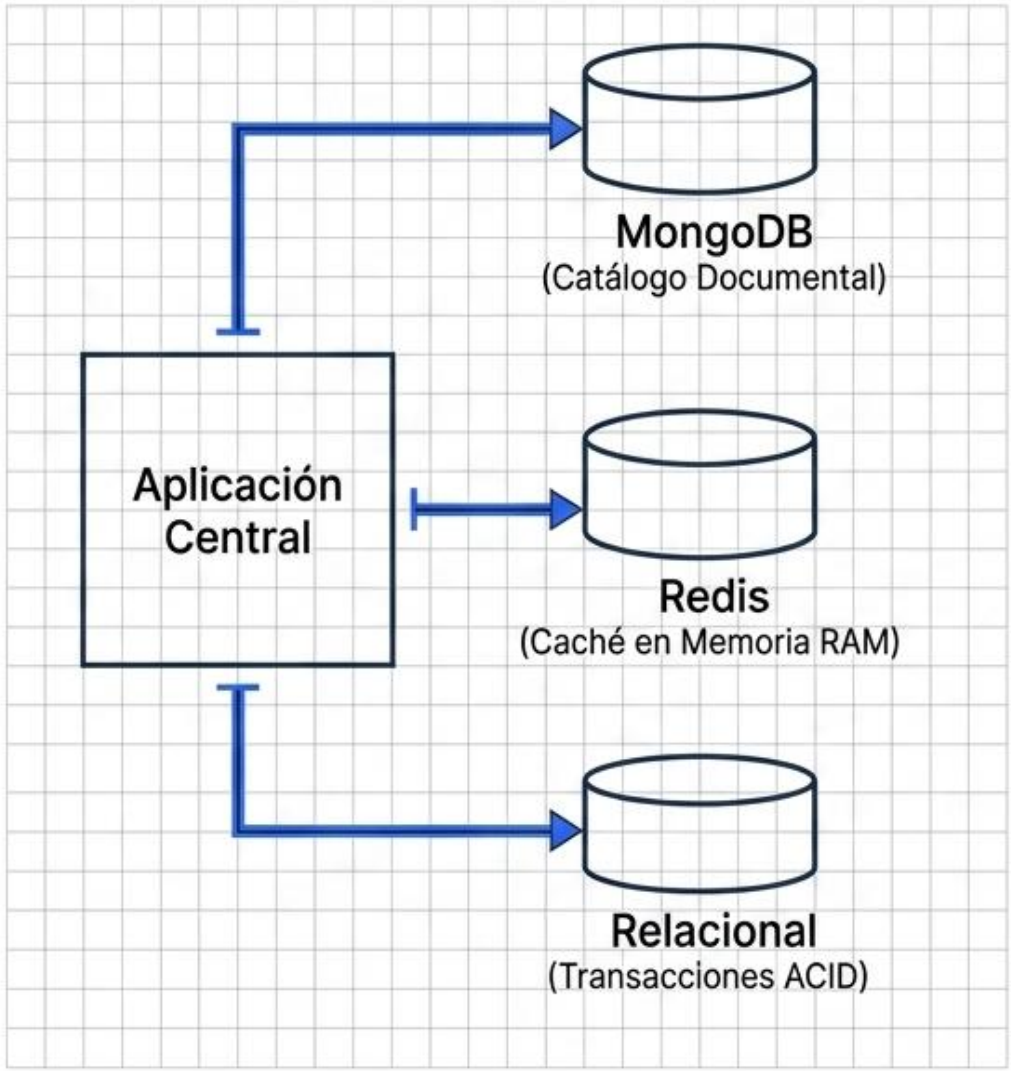
Distribución:
Arquitectura Peer-to-Peer descentralizada donde todos los nodos cumplen roles idénticos (sin punto único de falla).



Comportamiento CAP:
Tiende a AP. Nativamente disponible. Excelente para recibir avalanchas de escrituras continuas tolerando consistencia eventual.



Redis y la persistencia políglota



Redis (Almacenamiento Clave-Valor en memoria)

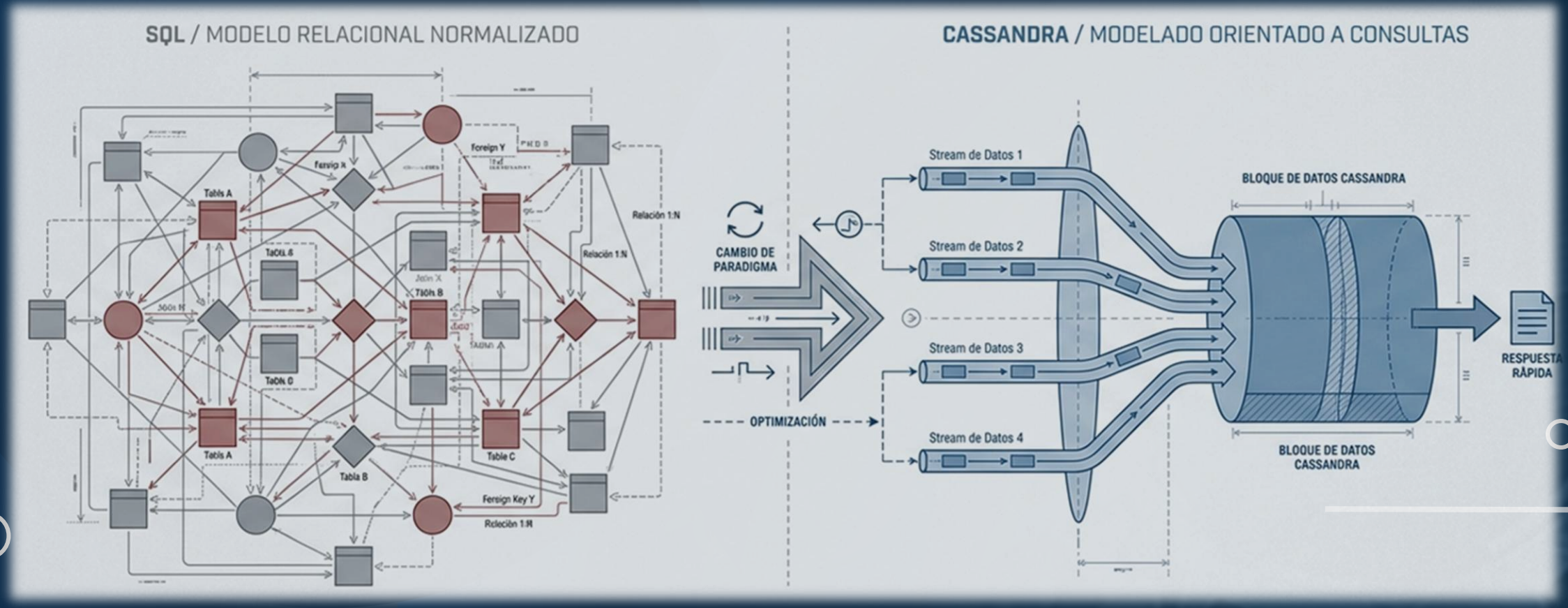
- **Arquitectura basada en RAM** diseñada para garantizar **latencia ultra-baja.**
- Escalamiento distribuido mediante **Redis Sentinel** o **Redis Cluster.**
- En el Teorema CAP, su prioridad es la **velocidad extrema** y la **disponibilidad** (AP), asumiendo consistencia eventual en cachés o sesiones de usuario.

Conclusión: La Arquitectura Políglota

En la industria moderna, ningún motor de base de datos soluciona todos los problemas. El diseño profesional combina **arquitecturas según la dimensión del flujo de datos:**

- Relacional: Transacciones estrictas y auditoría.
- Documental (MongoDB): Perfiles y catálogos flexibles.
- Distribuida AP (Cassandra): Ingestión masiva de eventos IoT.
- Memoria (Redis): Contadores en tiempo real y eliminación de latencia.

Modelado de Datos en Apache Cassandra



DE LA NORMALIZACIÓN RELACIONAL A LA EFICIENCIA ORIENTADA A CONSULTAS

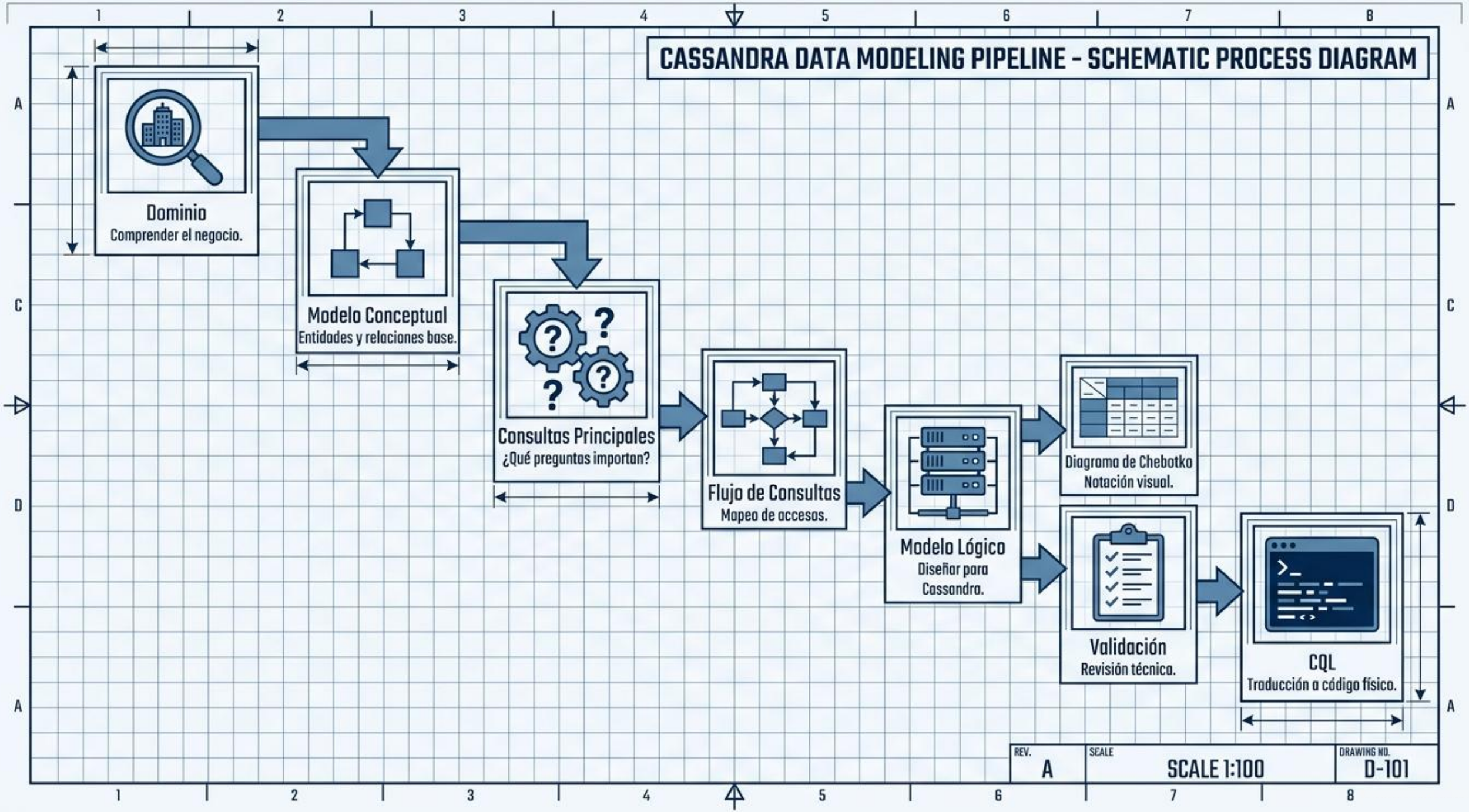
Diseño basado en almacenamiento

Diseño basado en consultas (Query-Driven)



Regla de Oro del Modelado en Cassandra:
Primero se define **CÓMO** se va a consultar la información.
Luego, se define cómo se va a almacenar.
El comando **CREATE TABLE** es una consecuencia del modelo, nunca el punto de partida.

CASSANDRA DATA MODELING PIPELINE - SCHEMATIC PROCESS DIAGRAM



Modelo de Chen (Referencia Conceptual)



ADVERTENCIA

Este modelo conceptual sirve exclusivamente para mapear y entender el problema de dominio.

No se debe trasladar directamente a tablas en Cassandra.

En bases de columnas anchas, normalizar este modelo para evitar redundancia es un antipatrón.

REV.	SCALE	DRAWING NO.
A	SCALE 1:100	D-101

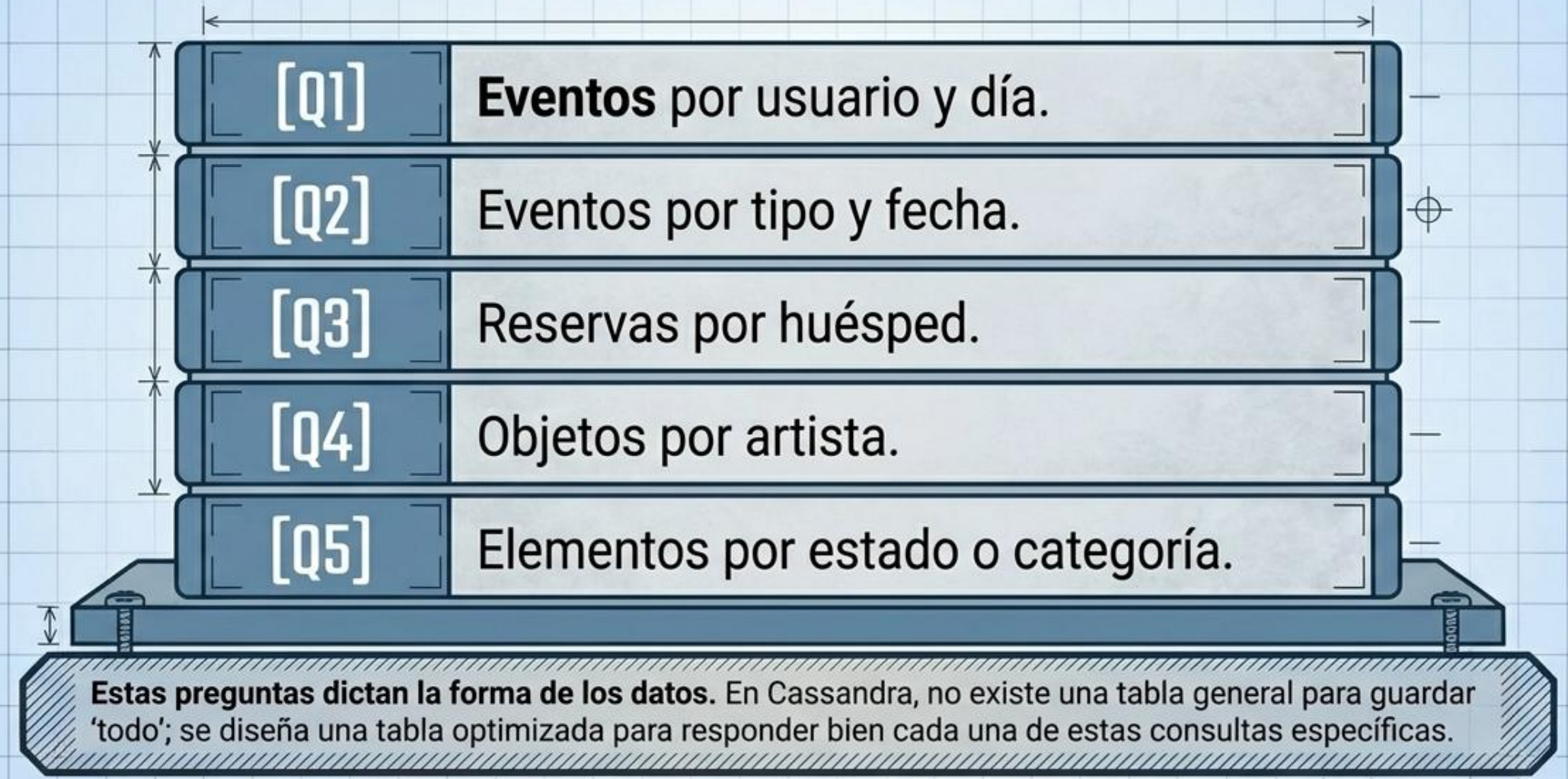
¿Por qué el modelo relacional fracasa aquí?

		Relacional	Cassandra
1	**Propósito**	Normalización, evitar redundancia	Desnormalización, optimizar lectura
2	**Relaciones**	Claves foráneas y Joins nativos	Una tabla por consulta, ausencia de Joins
3	**Flexibilidad**	Consultas ad-hoc dinámicas	Rutas de acceso predefinidas

¡Atención Arquitectónica! Dado que Cassandra no soporta Joins, intentar simularlos mediante cruces de datos del lado del cliente destruye el rendimiento. Si necesitas cruzar datos, necesitas desnormalizar en una nueva tabla.

REV. A | SCALE 1:100 | DRAWING NO. D-101 | TYLE.

Las consultas como cimientos del sistema



Estas preguntas dictan la forma de los datos. En Cassandra, no existe una tabla general para guardar 'todo'; se diseña una tabla optimizada para responder bien cada una de estas consultas específicas.

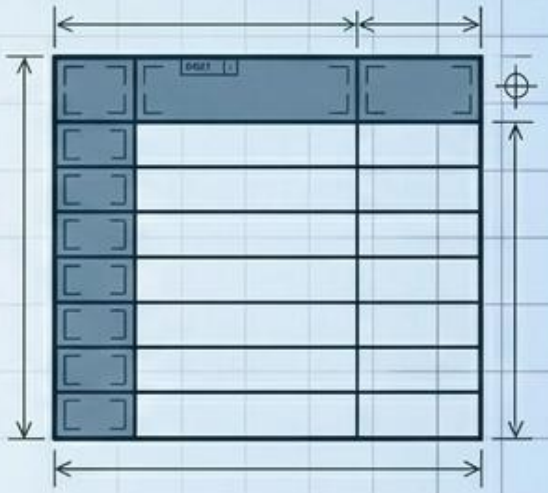
REV.	SCALE	DRAWING NO.	TYLE.
A	1:100	D-102	

El proceso de modelado: De la consulta al esquema

Consulta Q1

Para cada consulta se debe definir:

- Nombre de la tabla
- Clave de Partición
- Clave de Clustering
- Columnas devueltas
- Orden de lectura



Esquema optimizado.

REV.	SCALE	DRAWING NO.	TYLE.
A	1:100	D-103	

La clave primaria compuesta: el núcleo del esquema

Partition Key (Clave de Partición)

Determina el agrupamiento físico en los nodos (el "fichero").

Responde a la pregunta: ¿Con qué dato exacto empieza la búsqueda?

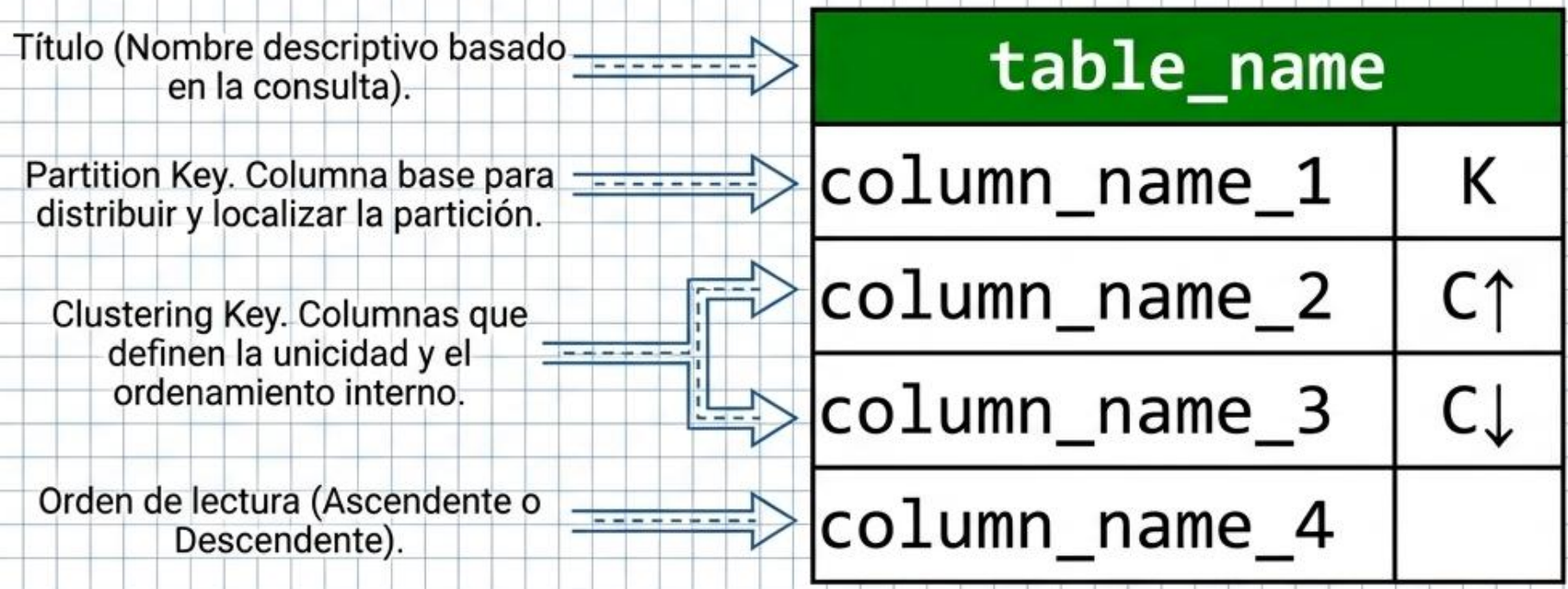


Clustering Key (Clave de Agrupamiento)

Determina el ordenamiento y diferenciación lógica interna dentro de esa partición.

REV.	SCALE	DRAWING NO.	TYLE.
A	1:100	D-104	

LEYENDA DE NOTACIÓN CHEBOTKO



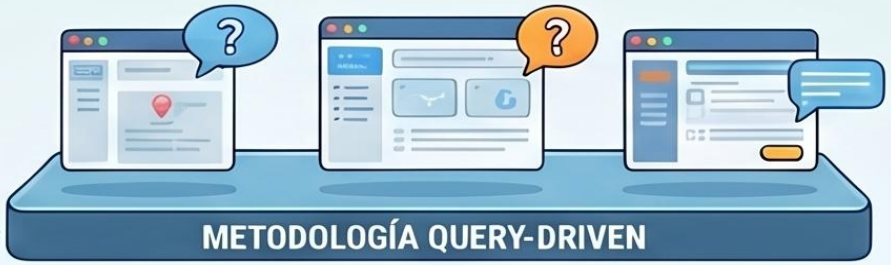
D-105	
1:100	A

IDD-II

Modelado de Datos en Cassandra: La Metodología Chebotko

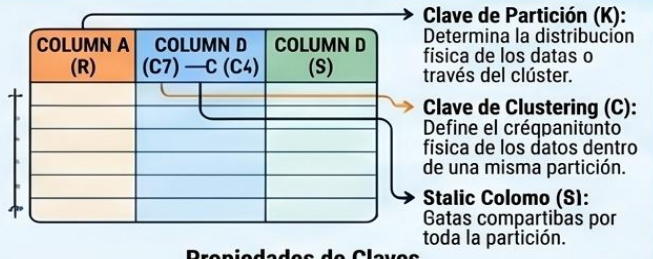
CAPA DE DOMINIO CONCEPTUAL
(Consultas de la Aplicación)

La **Consulta** es el **Punto de Partida**.
Se diseñan tablas especificas para responder preguntas concretas de la interfaz de usuario.



CAPA DE MODELADO LÓGICO
(Diagramas de Chebotko)

ANATOMÍA DEL DIAGRAMA DE CHEBOTKO



Propiedades de Claves

Componente	Siglo	Función Principal
Perthion Key	K	Distribuye datos (Squlóbrin)
Clustering Key	C↑ / C↓	Ordens datos internamente (holosidad)
Static Column	S	Datos compartidos por pertición

Q1: ¿Qué hoteles hay cerca de un punto de interés?

Table: hotels_by_poi (Q1)

poi_name (R)	hotel_id (C1)	hotel_name	address
…	…	…	…
…	…	…	…

K: poi_name | C↑: hotel_id
Permite boscer hoteles cerca de un punto de interés.

Tabla: hotels_by_poi (Q1)

poi_name (K)	hotel_id (C↑)	hotel_name	address
…	…	…	…

K: port_name | C↑: hotel_id
Permite buseer hoteles cerea de un punto do punto de interós.

Q4: ¿Qué habitaciones están disponibles en un hotel por fecha?

Table: available_rooms_by_hotel_date (Q4)

hotel_id (K)	date (C↑)	room_type	prise
…	…	…	le_anallabte
…	…	…	…

K: hotel_id | C↑: date
Optimiza la búsqueda de disponibilidad por rango de fechas.

Q7: ¿Qué reservas tiene un huésped por apellido?

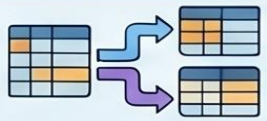
Tabla: reservations_by_guest (Q7)

guest_last_name (K)	hotel_id (C↑)	reservation_id
…	…	…
…	…	…

K: guest_last_name | C↑: hotel_id
Permito localizar rásprvas por el nombro del huésped.

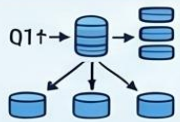
PRINCIPIOS CLAVE DE DISEÑO

Desnormalización Total

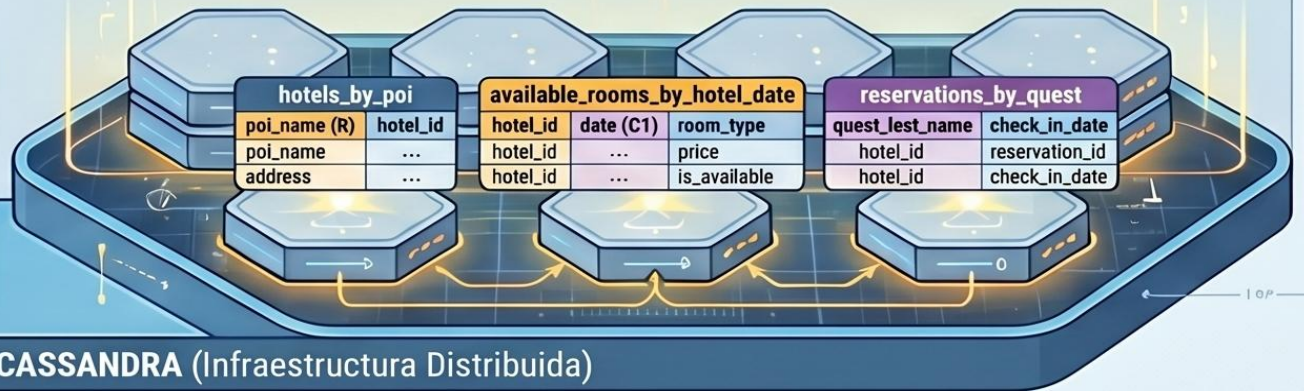


Se evita el uso de Joins duplicando datos en tablas optimizadas para el rendimiento.

Minimización de Particiones



El objetivo principal es reducir el número de nodos consultados para cada respuesta.



CLÚSTER DE CASSANDRA (Infraestructura Distribuida)

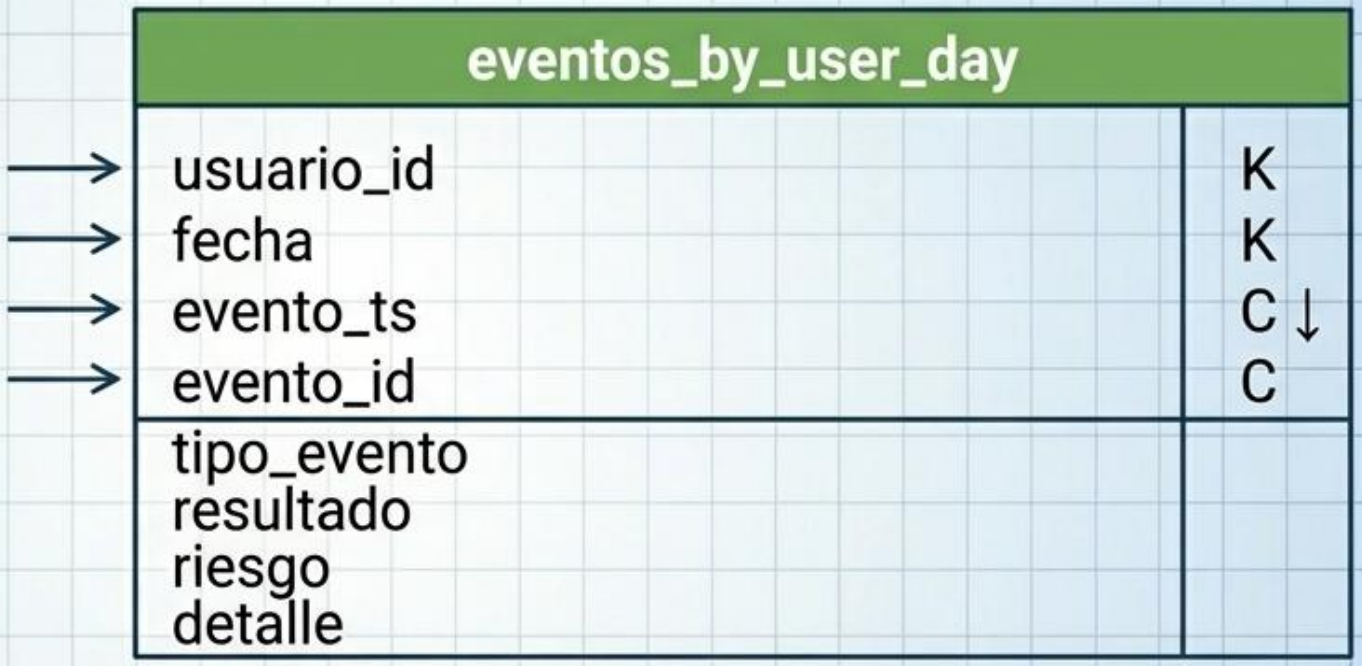
UADE

La clave primaria compuesta: el núcleo del esquema

REQUISITO (HUMANO)

Consulta: Consultar eventos de un usuario en una fecha, ordenados del más reciente al más antiguo.

DIAGRAMA CHEBOTKO (TÉCNICO)



Las particiones agrupan al usuario por día (K); el tiempo (C↓) garantiza que lo más reciente aparezca primero.

REV. A | SCALE 1:100 | DRAWING NO. D-106 | TYLE.

LEYENDA DE NOTACIÓN CHEBOTKO (EXTENDIDA)

Traducción Física

K	→ Se mapea a Clave de Partición en CQL.	**C↓**	→ Se mapea a **WITH CLUSTERING ORDER BY** (columna DESC).

Checklist de Validación Arquitectónica

- [✓] ¿La consulta siempre provee la clave de partición completa?
- [✓] ¿El clustering key responde naturalmente al orden requerido?
- [✓] ¿La partición tiene un tamaño predecible y razonable?
- [✓] ¿El diseño evita depender del modificador ALLOW FILTERING?
- [✓] ¿El modelo evita requerir simulaciones de joins en la aplicación?
- [✓] ¿La duplicación de datos entre tablas está justificada por la velocidad de lectura?

Cierre

Un buen modelo Cassandra no representa todas las relaciones posibles; representa las rutas de consulta críticas de forma eficiente.

REV. A | SCALE 1:100 | DRAWING NO D-107 | TYLE.



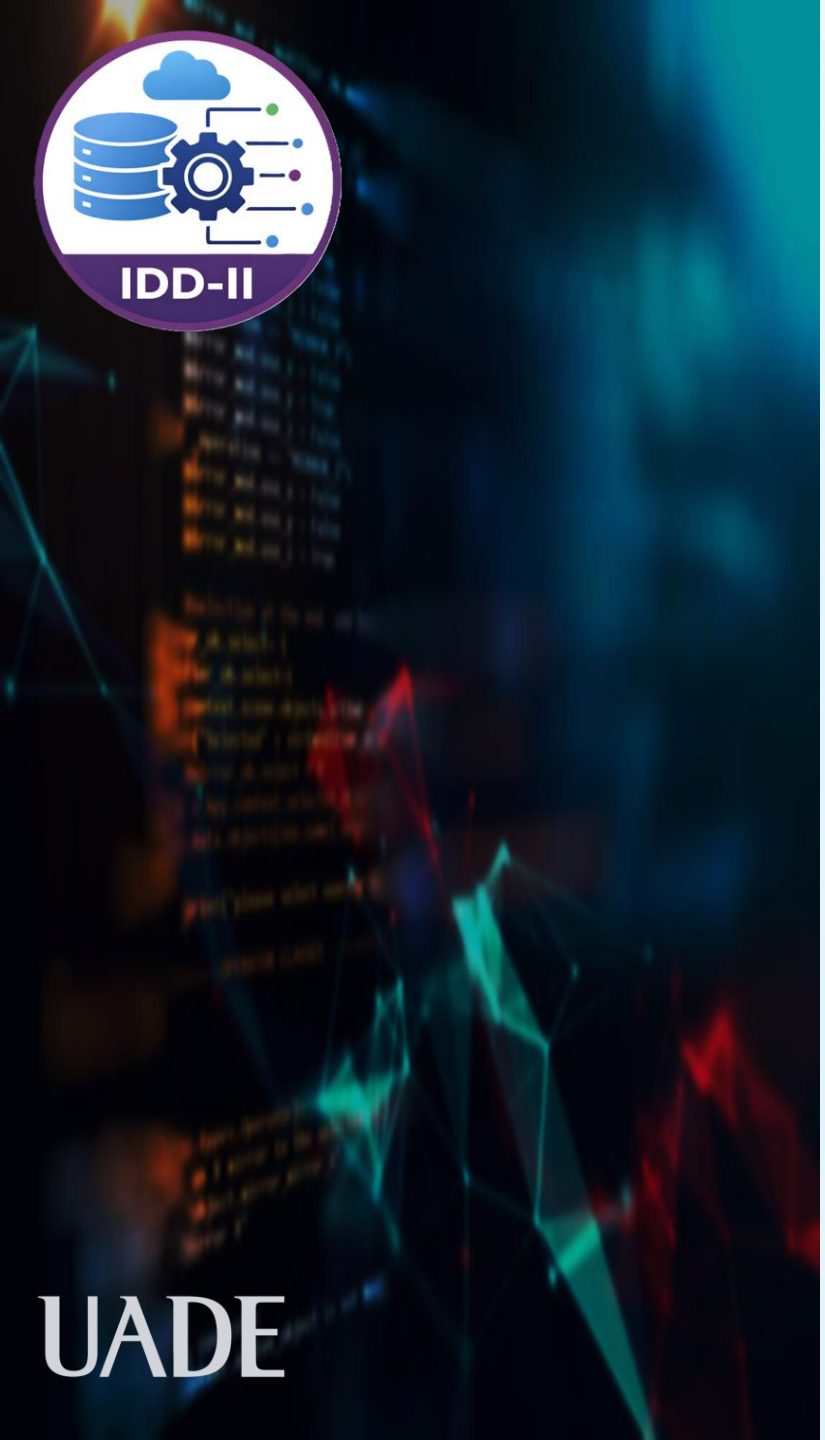
Actividad

Laboratorio de modelado Cassandra
Plataforma de eventos digitales

Una empresa de comercio digital registra eventos generados por sus usuarios en la plataforma. Cada evento representa una acción ocurrida dentro del sitio o la aplicación móvil: inicio de sesión, búsqueda de producto, visualización de producto, agregado al carrito, intento de pago, pago rechazado, cambio de contraseña, acceso desde dispositivo nuevo o evento marcado como sospechoso.

La empresa necesita almacenar estos eventos en Cassandra porque el volumen crece rápidamente, las escrituras son constantes y las consultas más importantes ya están identificadas. No se busca construir un modelo relacional normalizado, sino un modelo Cassandra orientado a consultas.

UADE



PROXIMA CLASE

Clase 9 - Persistencia de objetos e integración con aplicaciones

Tema:

Modelos de replicación:

- Master–Slave
- Peer to Peer
- Quorum

Tipos de consistencia:

- Eventual
- Plena de lectura / escritura

Bases de datos orientadas a objetos (conceptual).

Persistencia de objetos (introducción).